Assignment 3

Software Development and Integration

Analytical report and practical project: the fuzzy-scorecard repository with a GitHub Actions CI/CD pipeline

Case project: Fuzzy Rules-Based Mortgage Loan Valuation Model (Master's thesis)

Continuation of Assignments 1 and 2 (project plan; methodology and architecture of the same project)

Alibek Asset, Astana IT University

Repository: https://github.com/Gontafi/fuzzy-scorecard

October 2026

Contents

Introduction.....3

1. Stages and organisation of the software development process.....3

2. Criteria for selecting technologies and tools.....4

3. Version control systems: Git and GitHub.....5

4. Continuous integration and continuous delivery.....6

5. Features of scientific software development.....6

6. Practical part: the fuzzy-scorecard repository.....7

6.1 Purpose and design of the module.....7

6.2 Justification of the chosen technologies.....8

6.3 Use of Git and GitHub.....9

6.4 The CI/CD pipeline.....9

6.5 Automated testing and code quality.....10

6.6 Result of the demo run.....10

7. Conclusion and challenges.....11

References.....12

Introduction

In Assignment 1 I planned my Master's thesis project, "Fuzzy Rules-Based Mortgage Loan Valuation Model", and in Assignment 2 I evaluated the methodology (CRISP-ML(Q) gates with Kanban iterations) and the architecture (a batch pipeline with a model library and a shared evaluation harness) of the same project. This report covers the third topic of the course: how the development of such software is organised in practice, how the tools are chosen, and how version control and continuous integration keep the code trustworthy.

The report has two parts. The theoretical part (Sections 1 to 5) reviews the stages of the software development process, the criteria for choosing technologies, the version control systems, the practices of continuous integration and continuous delivery (CI/CD), and the features that distinguish scientific software from ordinary business software. The practical part (Section 6) describes a small module that I extracted from the thesis code and published as an open repository, fuzzy-scorecard, together with its GitHub Actions pipeline, tests and documentation. Section 7 contains conclusions and the difficulties I met.

In Assignment 2 I noted that the thesis code lived in a private repository without an automated test run, which I listed as a risk for reproducibility (NFR3) and for single-person knowledge (R3). This assignment gave me a reason to fix part of that: the mathematical core of the model, the monotone fuzzy scorecard, is now a separately installable package with tests that run on every change.

1. Stages and organisation of the software development process

A software development process is the set of activities that turn a need into working, maintained software. Every process model contains the same activities: requirements analysis, design, implementation, verification, delivery and maintenance. They differ in the order of these activities and in how often the cycle repeats. In the waterfall model the activities follow each other once, which is suitable when the requirements are stable and the cost of a change late in the project is accepted. In iterative and incremental models the cycle is repeated in short iterations, each of which produces a usable increment. Agile methods such as Scrum and Kanban are iterative models with specific rules about roles, meetings and work-in-progress limits. For research software the most useful description is the one given by Studer et al. (2021) in CRISP-ML(Q), where the phases of a machine-learning project are separated by quality gates, since in such projects the main uncertainty is whether the model will work at all.

Independently of the model, the modern development process is organised around a repository. The repository holds the code, the tests, the documentation and the configuration of the automation, and every change goes through the same path: a branch, a commit, a review and a

merge. Anyone can see what was changed, when, by whom and why, and the automation checks every change in the same way. Humble and Farley (2010) describe this organisation as a deployment pipeline, in which every change moves through a fixed sequence of automated stages before it is allowed to reach users.

Even for a project with one developer, as in my thesis, it helps to separate the roles of author and reviewer in time: a change is written on a branch and reviewed as a pull request before it is merged. The branching model should be as simple as the team allows. For a small team the trunk-based model with short-lived feature branches is sufficient, and Forsgren, Humble and Kim (2018) report that teams that merge small changes often perform better on both speed and stability than teams that keep long-lived branches.

Stage	Main activities	Artefact in the repository
Requirements	Understand the research question, agree thresholds and data	README, issue tracker, design notes
Design	Choose the module boundaries and the interfaces	Package layout, docstrings, pyproject.toml
Implementation	Write the code on a branch in small commits	Source files, commit history
Verification	Unit tests, lint, type and format checks, code review	tests/, CI workflow, pull request
Delivery	Version, tag, build the package, publish the release	CHANGELOG, tags, release workflow
Maintenance	Dependency updates, bug fixes, documentation	Dependabot, issues, new releases

Table 1. Stages of the development process and the artefacts that represent them in a modern repository.

2. Criteria for selecting technologies and tools

The choice of a language, a framework or a library is a decision with long-term consequences, so it needs explicit criteria, otherwise habit decides. From the lectures and from my own experience in the thesis project I use six criteria. The first is fitness for the task: does the technology solve the problem directly, with a small amount of glue code? The second is the ecosystem: are there mature libraries for the neighbouring tasks, such as numerical optimisation, data loading and plotting? The third is the skills of the team and of the people who will read the code later. In the thesis project these are the supervisor, the reviewers of the papers and, possibly, the model-risk team of a bank. The fourth is maintenance cost: the number of dependencies, the frequency of breaking changes, the availability of long-term support. The fifth is portability and reproducibility: whether the same environment can be recreated on another machine in a year. The sixth is licensing: whether the licence of a dependency allows publication of the code and use in a bank.

These criteria often conflict. A compiled language would give better run time for the bootstrap evaluation in the thesis, but it would make the code unreadable for the audience and add build steps that break reproducibility. I find it useful to write the decision down together with the

alternatives, so that it can be revisited when the context changes. Section 6.2 does this for the practical module.

3. Version control systems: Git and GitHub

A version control system (VCS) stores the history of a set of files, lets several people work on them at the same time and merges their changes. The first generation of tools (RCS, CVS, Subversion) used a central server that held the history; a developer checked out a working copy and committed back to the server. The second generation, represented by Git and Mercurial, is distributed: every clone contains the full history, and commits are made locally and exchanged with other clones afterwards (Chacon and Straub, 2014). This design makes commits and branches cheap and allows work without a network connection, which matters for a researcher who runs long experiments on a workstation.

Git is by now the dominant system. The Stack Overflow developer survey of 2022 reported that about 94 percent of professional developers use Git, and the figure for research software is similar. Its model is simple once understood: a commit is a snapshot of the whole tree identified by a hash, a branch is a movable pointer to a commit, and a merge creates a commit with two parents; rebasing, cherry-picking and tagging are built from these objects.

GitHub is a hosting platform built around Git. It adds what Git itself does not have: user accounts and permissions, pull requests with code review, an issue tracker, wikis, releases with attached files, and GitHub Actions for automation. GitLab offers an equivalent set of features and can be installed on one’s own servers, which some universities and banks prefer for data-protection reasons. Bitbucket is close to the Atlassian tools (Jira, Confluence). Table 2 compares the options against the criteria of Section 2. For a public research package I chose GitHub, because reviewers and other researchers will look for the code there, it is free for public repositories, and its CI minutes are free for them as well. The thesis data pipeline, which must stay private because of the licence of the raw files, lives in a separate private repository, and the same workflow files could be moved to GitLab without changes to the code.

Criterion	Git (tool)	GitHub	GitLab	Subversion
Model	Distributed	Hosting for Git	Hosting for Git	Centralised
Offline work	Yes	Yes (via Git)	Yes (via Git)	No
Code review	External	Pull requests	Merge requests	External tools
CI/CD	External	GitHub Actions	GitLab CI, self-hosted runners	External (Jenkins)
Self-hosting	Yes	Enterprise only	Yes, free edition	Yes
Cost for a public project	Free	Free, incl. CI	Free, limited CI	Free

Criterion	Git (tool)	GitHub	GitLab	Subversion
Community of scientists	Universal	Largest	Common in institutes	Legacy

Table 2. Comparison of version control tools and platforms against the criteria of Section 2.

4. Continuous integration and continuous delivery

Continuous integration (CI) is the practice of merging every developer’s work into the main line at least once a day and verifying each merge by an automated build and test run (Fowler, 2006). The purpose is to find integration problems while they are small. Three rules make CI work. First, the build must be fully automated and start from a clean machine, so that “it works on my computer” is not an acceptable state. Second, the build must be fast, in the order of minutes, or developers will stop waiting for it. Third, a broken build must be fixed before anything else is merged. A CI service such as GitHub Actions, GitLab CI or Jenkins runs the build on every push and reports the result next to the commit or pull request.

Continuous delivery (CD) extends the pipeline to the point where every change that passes the checks is packaged and can be released with one action, such as pushing a tag. Continuous deployment goes one step further and releases automatically. For a library the deliverable is a versioned package; for a service it is a container image or a deployed environment. Humble and Farley (2010) call the whole sequence a deployment pipeline and recommend building the artefact once and promoting that same artefact through the stages instead of rebuilding it each time.

Several practices go with CI/CD. Automated tests at several levels give the pipeline its value; without them it only checks that the code compiles. Static analysis and formatting tools catch a class of errors before the tests run. Dependency pinning and caching make the builds reproducible and fast. A matrix build runs the same tests across several versions of the language. The configuration of the pipeline is itself stored in the repository as code, so that changes to it are reviewed like any other change. Forsgren, Humble and Kim (2018) summarise the evidence that these practices are associated with higher deployment frequency and lower change failure rates.

5. Features of scientific software development

Scientific software differs from commercial software in its purpose and in who writes it. Its purpose is usually a result for a paper or a thesis; it rarely has to serve users continuously. It is often written by researchers who were not trained as software engineers, under time pressure, with requirements that change as the research progresses. Wilson et al. (2014) list the consequences: code that is hard to reproduce, undocumented, untested and dependent on one person. The same authors propose a set of “best practices for scientific computing”, of which the most important are to write programs for people and not for computers, to automate repetitive tasks, to make

incremental changes under version control, to plan for mistakes with tests and assertions, and to document the design and purpose, not the mechanics.

The requirements of scientific software differ slightly from the usual list. Reproducibility is a hard requirement: another researcher must be able to obtain the same numbers from the same data and code. This is achieved by fixing random seeds, by pinning the versions of dependencies, by storing the environment definition in the repository and by running everything from a single command. Correctness is harder to check than in business software, because the right answer is unknown, so tests check mathematical properties (memberships sum to one, the score is monotone, the explanation sums to the logit) instead of exact outputs. Performance matters at the scale of the experiment, which for my thesis is 1.4 million loans and twenty bootstrap refits in under an hour. Portability matters because reviewers and collaborators use different machines.

The environment of scientific Python is well suited to these requirements. Virtual environments isolate the dependencies of a project and pyproject.toml declares them; NumPy and SciPy provide the numerical core, pandas the tables, scikit-learn the baselines and metrics, matplotlib the figures and pytest the property tests. Jupyter notebooks are useful for exploration but should not be the place where results are produced. The thesis project follows this division, and the next section applies the same practices to a small, public piece of its code.

6. Practical part: the fuzzy-scorecard repository

6.1 Purpose and design of the module

The repository https://github.com/Gontafi/fuzzy-scorecard contains the Python package fuzzyscore. It implements the mathematical core of the thesis model in a form that does not depend on the private data pipeline. The package builds a Ruspini partition on every numeric feature, that is, a family of triangular membership functions that sum to one at every point, from a list of knots chosen by the analyst, for example the underwriting thresholds of FICO, loan-to-value and debt-to-income ratios. The partition is converted into cumulative “at least” terms oriented in the agreed risk direction, and a logistic scorecard is fitted on these terms with an L1 penalty and non-negative weights. Because the weights cannot be negative, the score is monotone in every feature by construction, the property that a credit officer and a regulator expect. The package also explains any single score as a list of activated terms with their contributions, and computes ROC AUC, the Brier score and the expected calibration error.

The module has five source files and a test directory (Figure 1). The partition and metric functions are pure NumPy functions with no state. The model is a small class with the methods fit, predict_proba, explain and to_table; the optimisation uses the bound-constrained L-BFGS-B

method from SciPy, which handles the non-negativity directly, and because the weights are non-negative the L1 penalty becomes a linear term and the objective stays smooth. A synthetic loan generator provides data for tests, examples and the CI smoke test, so that the pipeline does not depend on any external download. A command-line entry point, fuzzyscore demo, runs the whole chain and writes a JSON report.

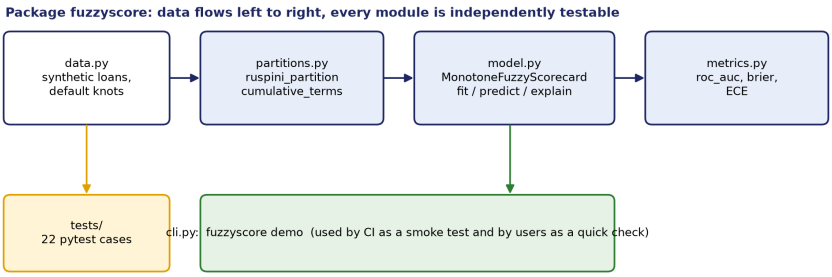


Figure 1. Modules of the fuzzyscore package and the data flow between them.

6.2 Justification of the chosen technologies

I applied the criteria of Section 2 to each choice. The language is Python 3.10 and later. It is the language of the thesis code, of the reviewers and of most model-risk teams, and it has the libraries needed for the neighbouring tasks. The alternative, R, would have been equally readable to statisticians but would have split the thesis code base into two languages. The numerical core uses NumPy and SciPy only. They are the most stable packages in the scientific Python ecosystem, have no compiled dependencies that the user must build, and SciPy already contains the bound-constrained optimiser I need; a custom projected-gradient solver would have meant more code to test for no gain. I did not add pandas or scikit-learn as hard dependencies, because the package works on plain arrays and a smaller dependency set installs faster in CI and is easier to audit.

For testing I use pytest, the de facto standard in Python, because its fixtures and parametrisation express property tests compactly, and pytest-cov to enforce a coverage gate of 90 percent. For static checks I use ruff, a single fast tool that replaces flake8, isort and black and runs in under a second, which keeps the first stage of the pipeline short. Packaging uses pyproject.toml with setuptools, the standard that pip, build and GitHub Actions all understand without extra configuration. Git is the version control system for the reasons of Section 3, GitHub is the host because the package is public and the CI minutes are free, and GitHub Actions is the CI service because it needs no additional account or server and its configuration is a file in the repository. Table 3 records the alternatives considered.

Decision	Alternatives considered	Reason for the choice
Python 3.10+	R, Julia	Same language as the thesis code; readable for the audience; largest ecosystem

Decision	Alternatives considered	Reason for the choice
NumPy + SciPy only	scikit-learn, cvxpy, PyTorch	Smallest stable dependency set; SciPy L-BFGS-B handles the bounds; faster CI install
pytest + pytest-cov	unittest	Compact property tests, fixtures, parametrisation, coverage gate
ruff	flake8 + isort + black	One tool, one configuration, sub-second run
setuptools via pyproject.toml	poetry, hatch	Standard, no extra tool for users or CI
GitHub + GitHub Actions	GitLab CI, Jenkins	Public hosting, free CI, no server to maintain, reviewers already use it
MIT licence	Apache 2.0, GPL	Simplest permissive licence, compatible with use in a bank

Table 3. Technology decisions for the module with the alternatives that were considered.

6.3 Use of Git and GitHub

The repository was created locally with a main branch and seven commits that follow the order of the work: packaging skeleton with licence and README; partitions, metrics and the data generator; the model and the command line; the test suite; the CI configuration; the contributing guide with the changelog; and this report in the docs directory. Each commit is a self-contained step that passes the tests, so that the history can be read and bisected. The release 0.1.0 is marked with an annotated tag. The repository is published publicly on GitHub under my account, with the default branch main.

On the GitHub side the repository contains the elements asked for in the assignment. The README explains what the package does, how to install it and how to use it, and shows the CI status badge. The LICENSE file holds the MIT licence. The .github directory contains issue templates for bug reports and feature requests, a pull request template with a checklist that mirrors the CI stages, and a Dependabot configuration that opens monthly pull requests for outdated actions and Python packages. A CONTRIBUTING file describes the branch naming, the local checks and the release procedure, and a CHANGELOG records the versions. Branch protection on main requires the CI check to pass before a merge, which enforces the rule that a broken build is fixed first.

6.4 The CI/CD pipeline

The pipeline is defined in two workflow files (Figure 2). The first, ci.yml in the .github/workflows directory, runs on every push and every pull request to main and has three dependent jobs. The lint job installs ruff and runs the lint rules and the format check; it finishes in about twenty seconds and stops the pipeline early when the code does not meet the style rules. The test job runs after the lint job, on a matrix of Python 3.10, 3.11 and 3.12, installs the package in editable mode with the development extras, runs pytest with the coverage gate, then runs the command fuzzyscore demo as a smoke test of the installed entry point, and uploads the JUnit report and the demo report as

artefacts. The build job runs after the tests and builds the source and wheel distributions, which are uploaded as an artefact so that the same package that was tested can be inspected or installed.

The second file, release.yml in the same directory, implements continuous delivery. It is triggered by a tag of the form v*, runs the tests again, builds the distributions and creates a GitHub Release with the built files attached and release notes generated from the merged pull requests. Pushing a tag is the only manual action needed to publish a version. The workflow has write permission on the repository contents only, and the CI workflow is read-only, following the principle of least privilege. The complete workflow files are in the repository at https://github.com/Gontafi/fuzzy-scorecard/blob/main/.github/workflows/ci.yml and https://github.com/Gontafi/fuzzy-scorecard/blob/main/.github/workflows/release.yml; the runs are visible at https://github.com/Gontafi/fuzzy-scorecard/actions.

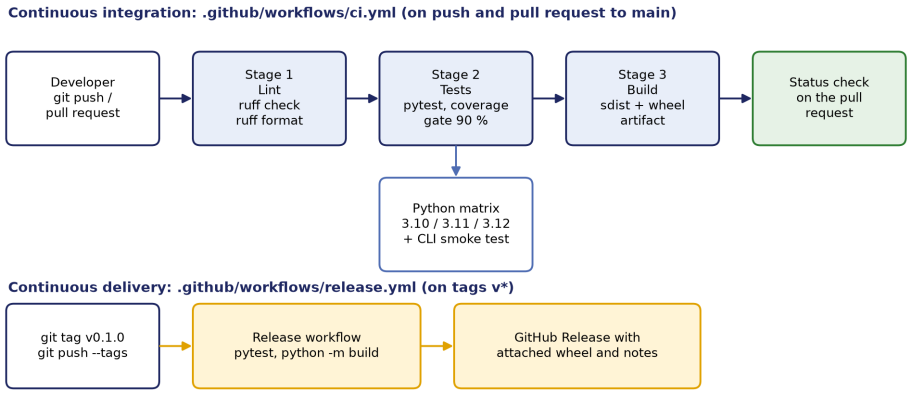


Figure 2. The CI pipeline (top) and the release pipeline (bottom) of the repository.

6.5 Automated testing and code quality

The test suite contains 22 cases in four files and runs in under one second. Following the advice of Section 5, most tests check mathematical properties instead of fixed outputs. For the partitions: memberships are non-negative and sum to one on a fine grid, values outside the knots are clipped to the end terms, cumulative terms are non-decreasing in the feature, and invalid knots raise an error. For the metrics: the rank-based AUC equals the pairwise definition on random data, gives one, zero and one half in the perfect, inverted and constant cases, and refuses a single-class input; the Brier score and the calibration error are zero for perfectly calibrated inputs. For the model: the fitted weights are non-negative, the score is monotone in each of the three features on a grid of 25 values for 200 loans, the explanation of a single loan sums exactly to its logit, a stronger L1 penalty gives a shorter scorecard, and the out-of-sample AUC on fresh synthetic loans is above 0.72. For the command line: the demo report contains the metrics and the JSON file is written. Coverage measured by pytest-cov is 97 percent of the statements, above the 90 percent gate.

6.6 Result of the demo run

Figure 3 shows the scorecard produced by fuzzyscore demo with 20,000 synthetic loans in each of the training and test samples. The fitted scorecard has ten active terms out of thirteen candidates; the terms with zero weight were removed by the L1 penalty. The weights are all positive and oriented as expected: a lower credit score, a higher loan-to-value ratio and a higher debt-to-income ratio each add risk. On the held-out sample the AUC is 0.787, the Brier score is 0.032 and the expected calibration error is 0.003 at a default rate of 3.5 percent. These numbers are for synthetic data and only show that the pipeline works end to end; results on the real Freddie Mac data are reported in the thesis papers. The same JSON report is produced in every CI run and stored as an artefact, which gives a simple regression check between runs.

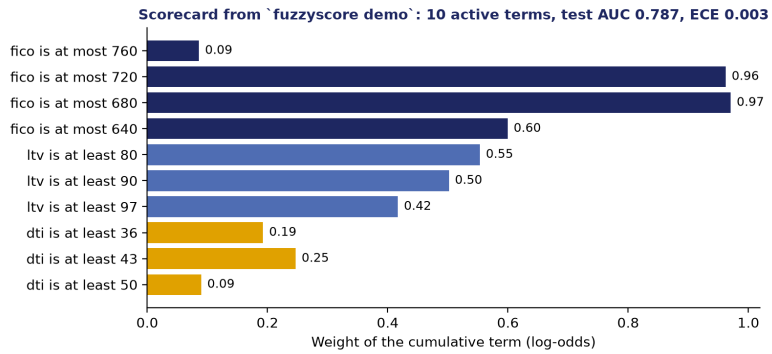


Figure 3. Weights of the active terms of the scorecard fitted by the demo command on synthetic loans.

7. Conclusion and challenges

The review in Sections 1 to 5 showed that the process models have more in common than they have differences: a repository as the single source of truth, small reviewed changes, automated checks on every change and a repeatable path to a release. For scientific software these practices address the weaknesses that the literature reports, poor reproducibility, missing tests and dependence on one person.

The practical result is the public repository fuzzy-scorecard with a tested package, a three-stage CI pipeline on three Python versions, a tag-driven release workflow, documentation, licence, templates and dependency updates. Compared with the state described in Assignment 2, the core model of the thesis can now be installed with one pip command, its central properties are verified on every change, and a reader of the papers can check the implementation without access to the private data. The work took about two days. Most of that time went into the tests and the documentation; the model code itself was ready on the first morning. Wilson et al. (2014) make the same observation about research code in general.

I met three difficulties. The first was deciding what to cut: the thesis code depends on the data pipeline, on pandas and on LightGBM baselines, and the module had to be small enough to test completely and to install in CI in under a minute, so I limited it to the partition, the fitting

procedure, the metrics and a synthetic generator, and left the real data and the baselines in the private repository. The second was the optimiser: the first version used an unconstrained solver with a soft penalty for negative weights, and the monotonicity test failed on a few grid points; replacing it with the bound-constrained L-BFGS-B method made the constraint exact and simplified the code. The third was the pipeline itself: the lint stage initially rejected code that the tests accepted (an unqualified zip and a quoted type annotation), a reminder that the two stages catch different classes of problems and that the linter configuration is a design decision to be committed and reviewed like the rest of the code.

The next steps are an interface for loading real loan files, an optional dependency on scikit-learn for the baselines, and publication on PyPI from the same release workflow. For the thesis project the more important consequence is procedural: the private data pipeline now has a template for its own CI configuration, and at the gate reviews with the supervisor a green pipeline can replace a notebook as evidence that the code works.

References

Chacon, S. and Straub, B. (2014). Pro Git, 2nd edition. Apress. Available at https://git-scm.com/book.

Forsgren, N., Humble, J. and Kim, G. (2018). Accelerate: The Science of Lean Software and DevOps. IT Revolution Press.

Fowler, M. (2006). Continuous Integration. martinfowler.com/articles/continuousIntegration.html.

Humble, J. and Farley, D. (2010). Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation. Addison-Wesley.

Stack Overflow (2022). Developer Survey 2022: Version control systems. survey.stackoverflow.co/2022.

Studer, S., Bui, T. B., Drescher, C., Hanuschkin, A., Winkler, L., Peters, S. and Müller, K.-R. (2021). Towards CRISP-ML(Q): A machine learning process model with quality assurance methodology. Machine Learning and Knowledge Extraction, 3(2), 392–413.

Wilson, G., Aruliah, D. A., Brown, C. T., Chue Hong, N. P., Davis, M., Guy, R. T., Haddock, S. H. D., Huff, K. D., Mitchell, I. M., Plumbley, M. D., Waugh, B., White, E. P. and Wilson, P. (2014). Best practices for scientific computing. PLoS Biology, 12(1), e1001745.

Asset, A. (2026). fuzzy-scorecard repository, https://github.com/Gontafi/fuzzy-scorecard.